



OPEN Modified grey wolf optimization for energy-efficient internet of things task scheduling in fog computing

Deafallah Alsadie^{1✉} & Musleh Alsulami²

Fog-cloud computing has emerged as a transformative paradigm for managing the growing demands of Internet of Things (IoT) applications, where efficient task scheduling is crucial for optimizing system performance. However, existing task scheduling methods often struggle to balance makespan minimization and energy efficiency in dynamic and resource-constrained fog-cloud environments. Addressing this gap, this paper introduces a novel Task Scheduling algorithm based on a modified Grey Wolf Optimization approach (TS-GWO), tailored specifically for IoT requests in fog-cloud systems. The proposed TS-GWO incorporates innovative operators to enhance exploration and exploitation capabilities, enabling the identification of optimal scheduling solutions. Extensive evaluations using both synthetic and real-world datasets, such as NASA Ames iPSC and HPC2N workloads, demonstrate the superior performance of TS-GWO over established metaheuristic methods. Notably, TS-GWO achieves improvements in makespan by up to 46.15% and reductions in energy consumption by up to 28.57%. These results highlight the potential of TS-GWO to effectively address task scheduling challenges in fog-cloud environments, paving the way for its application in broader optimization tasks.

Keywords Grey wolf optimizer (GWO), Fog-cloud computing, Task scheduling, Internet of things (IoT), Metaheuristic algorithm

In the contemporary landscape, the expansion of telecommunication networks profoundly influences the ongoing Internet of Things (IoT) revolution, which continues to gain momentum. Terminal devices assume a pivotal role in environmental sensing and data generation. However, due to their inherent limitations, these devices often resort to transmitting data to the cloud for storage, processing, analysis, and decision-making, owing to its robust computational and storage capabilities. Despite the ubiquitous accessibility of cloud computing resources, it falls short in addressing the latency requirements of latency-sensitive IoT applications¹. This underscores the complementary role of fog computing vis-a-vis the cloud, rather than serving as a substitute. The collaboration between cloud and fog computing is crucial to accommodate the diverse task lengths and computational demands. The proliferation of edge-cloud computing has led to a surge in user requests, which vary between cloud and fog nodes based on factors like task length, task sensitivity, and performance metrics such as delay, makespan, cost, and energy consumption². However, the unique characteristics of fog and cloud computing, coupled with the unpredictable nature of user requests and resource constraints, render task scheduling optimization a complex and indispensable topic worthy of discussion.

Task scheduling in fog-cloud computing systems plays a crucial role in optimizing performance and efficiently allocating computational resources^{1,3,4}. The emergence of fog computing, driven by the proliferation of Internet of Things (IoT) devices, offers decentralized computation by extending the cloud to the edge of the network. This proximity to IoT devices reduces latency and enhances response times compared to traditional cloud computing architectures^{4,5}.

Despite the benefits of fog computing, efficient task scheduling remains challenging due to factors such as unbalanced load distribution, dynamic network conditions, and varying resource availability^{6,7}. These challenges can lead to delays, resource underutilization, and inefficient energy consumption, underscoring the importance of developing practical task-scheduling algorithms.

To address these challenges, researchers have proposed various intelligent optimization techniques, including evolutionary algorithms⁷, ant colony optimization⁸, Q-learning², game theory⁹, and multi-objective evolutionary algorithms¹⁰. Each approach targets different aspects of task scheduling, such as optimizing time cost, task arrangement, reliability, and resource allocation. Moreover, meta-heuristic algorithms like the

¹Department of Computer Science and Artificial Intelligence, College of Computing, Umm Al-Qura University, 21961 Makkah, Saudi Arabia. ²Department of Software Engineering, College of Computing, Umm Al-Qura University, 21961 Makkah, Saudi Arabia. ✉email: dbsadie@uqu.edu.sa

Grey Wolf Optimizer (GWO) have gained attention for their ability to overcome the limitations of traditional optimization techniques^{11–13}. Inspired by the social behavior of grey wolves, GWO offers efficient exploration and exploitation of search spaces, leading to improved convergence and solution quality.

Expanding on this principle, our study aims to innovate task scheduling methods by improving the efficacy of the GWO algorithm within IoT settings. The proposed approach, named TS-GWO, generates a set of individuals representing task scheduling solutions, evaluates them using fitness values, and applies various operators to update the individuals iteratively until reaching stop conditions.

The primary objective of our research is twofold:

1. Introducing an innovative method for task scheduling within IoT environments through enhancements to the GWO algorithm.
2. Assessing the efficiency of the developed method through comprehensive experiments with various datasets and comparing its performance against well-known task scheduling techniques published in the literature.

Through our investigation, we aim to demonstrate the effectiveness of the TS-GWO method in optimizing makespan, task execution time, and energy consumption to meet the Quality of Service (QoS) requirements of IoT devices. Overall, our research contributes to advancing task scheduling techniques in fog-cloud computing systems, addressing critical challenges and improving system performance.

The subsequent sections of this paper are structured as follows: section “Related work” conducts a thorough review of existing literature in the field, identifying research gaps. The paper then proceeds to section “System model”, which describes the system model utilized in the study. Section “GWO algorithm fundamentals” offers a concise overview of the fundamentals of the GWO algorithm. In section “The proposed task scheduling with grey wolf algorithm”, we elaborate on the modified GWO algorithm designed for efficient task scheduling in fog-cloud computing environments. Section “Experiment and results” outlines the experimental setup, performance metrics, experimental results, and analysis. Finally, Section Conclusion presents the conclusions drawn from the study.

Related work

Task scheduling in fog-cloud computing systems is critical due to issues like unbalanced load and excessive resource utilization, leading to delays and overloads in cloud data centers⁶. Fog computing addresses these challenges by decentralizing processing, yet efficient task scheduling remains essential for optimal resource allocation. In IoT environments, where user demands are high, and fog computing platforms offer superior performance compared to cloud platforms, optimizing task allocation becomes crucial. To tackle the NP-hard complexity of multi-task scheduling, various intelligent optimization techniques have been proposed. Nguyen et al.⁷ introduced an evolutionary algorithm for IoT task scheduling in cloud-fog systems, aiming to balance time cost and task arrangement efficiently. Boveiri et al.⁸ proposed an enhanced ant colony optimization-based approach to optimize task order in multiprocessor environments. Tong et al.² developed a Q-learning task scheduling algorithm by combining neural network and Q-learning techniques to address IoT task scheduling in cloud systems. Yang et al.⁹ introduced a strategy based on game theory to improve the reliability of IoT task scheduling in cloud environments. Moreover, in a separate study, they presented a multi-objective evolutionary algorithm aimed at optimizing both performance time and resource allocation in fog computing environments in¹⁰.

Other approaches include⁵, who used visualization methods to optimize energy cost and delay in fog computing, and¹⁴, who utilized the moth-flame algorithm to optimize task scheduling in fog nodes for improved quality of service. Abualigah et al.¹⁵ introduced an intelligent task scheduling method based on the GWO to optimize time cost and resource allocation in cloud computing. Zeng et al.¹⁶ introduced a scheduling technique tailored for fog computing systems that cater to embedded systems. The primary aim was to reduce task execution time, thereby boosting user engagement. These studies collectively contribute to addressing the challenges of task scheduling in fog-cloud environments and supporting efficient resource utilization.

Various meta-heuristic algorithms face challenges such as randomness issues, limited global search capabilities, and convergence problems, often resulting in suboptimal solutions¹⁷. Researchers have turned to the GWO due to its effectiveness in overcoming these limitations¹⁸. Unlike Particle Swarm Optimization (PSO), which relies on individual particles to select the best solution, GWO uses the three best solutions to prevent convergence to local optima¹⁹. Moreover, GWO requires fewer parameters compared to Genetic Algorithms (GA), reducing complexity, computation time, and energy consumption¹⁹. GWO's dynamic nature makes it superior to Ant Colony Optimization (ACO), which does not account for the dynamic characteristics of computing resources. Introduced as a meta-heuristic approach by²⁰, GWO leverages swarm intelligence, mimicking the hierarchical structure and social interactions of grey wolves during hunting. Its benefits include superior convergence, minimal parameterization, and ease of implementation²¹. GWO has found applications in various domains such as optimization, classification, economic modeling, power dispatch, and vehicle routing²². With the growing interest in Multi-Objective Problems (MOP), particularly in cloud-fog computing, researchers aim to balance delay and energy consumption²³. Fog node execution reduces delay but increases energy consumption, while cloud execution reduces user energy consumption but increases delay. Finding a tradeoff strategy between delay and energy is crucial in cloud-fog environments, where task scheduling plays a vital role in processing IoT applications. Table 1 summarizes the studies as mentioned earlier.

Study	Technique	Focus	Advantages	Disadvantages	Target environment
7	Evolutionary algorithm	IoT task scheduling	Balances time cost and task arrangement	May have convergence issues	Cloud-fog systems
8	Enhanced ant colony optimization	Task order optimization	Efficient for multiprocessor environments	Prone to stagnation	Fog-cloud systems (multiprocessor)
2	Q-learning task scheduling	IoT task scheduling in cloud	Combines neural network and Q-learning for efficiency	High computational complexity	Cloud systems
9	Game strategy	Improve task reliability	N/A	Limited to reliability	Cloud environments
10	Multi-objective evolutionary algorithm	Performance time & Resource allocation	Optimizes both performance and resource usage	May require complex parameter tuning	Fog computing environments
5	Visualization-aided scheduling	Energy cost & delay optimization	User-friendly for design	Limited scalability for large systems	Fog computing
14	Moth-flame algorithm	Optimize task scheduling in fog nodes	Improves service quality and reduces execution time	May struggle with highly dynamic environments	Fog computing environments
11	Grey wolf optimizer	Time cost & resource allocation	Efficient and requires fewer parameters	May require modifications for specific problems	Cloud computing
16	Task scheduling for embedded systems	Minimize task execution time	Improves user experience	Limited to embedded systems	Fog computing (embedded systems)

Table 1. Comparison of task scheduling techniques in fog-cloud computing.

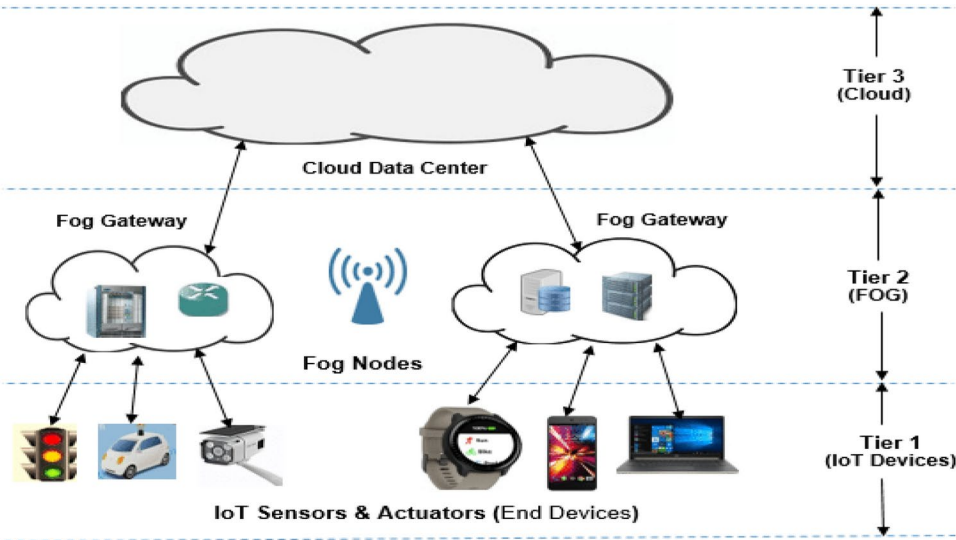


Fig. 1. Fog-cloud computing architecture.

System model

In a fog-cloud computing environment, the architecture is designed to handle the computational demands of IoT devices efficiently. It comprises fog nodes situated closer to the edge of the network, cloud servers located in centralized data centers, and interconnected IoT devices, as depicted in Fig. 1.

- **Fog nodes:** Fog nodes are intermediary computing devices deployed at the network edge, closer to the IoT devices. These nodes serve to preprocess and filter the data generated by IoT devices before transmitting it to the cloud. Fog nodes are equipped with moderate computational resources and storage capabilities, enabling them to execute tasks locally and reduce latency in data processing.
- **Cloud servers:** Cloud servers form the backbone of the fog-cloud architecture, providing vast computational resources and storage capacity. These servers are typically located in centralized data centers and are responsible for handling large-scale data processing tasks that cannot be efficiently executed at the edge. Cloud servers facilitate scalability and flexibility in managing diverse workloads from numerous IoT devices.
- **IoT devices:** IoT devices, ranging from sensors and actuators to smartphones and wearables, generate a continuous stream of data. These devices are interconnected with fog nodes and cloud servers to transmit data for processing and analysis. The data generated by IoT devices may vary in volume, velocity, and variety, necessitating adaptive and efficient data processing mechanisms.
- **Communication infrastructure:** The communication infrastructure in a fog-cloud environment enables seamless connectivity between IoT devices, fog nodes, and cloud servers. Various communication technologies, such as Wi-Fi, Bluetooth, and cellular networks, are utilized to establish reliable and low-latency connections.

tions. Additionally, protocols like MQTT (Message Queuing Telemetry Transport) and CoAP (Constrained Application Protocol) facilitate efficient data transmission and management.

- System interactions: In this environment, IoT devices interact with nearby fog nodes to transmit data for pre-processing and analysis. Fog nodes, equipped with computational capabilities, perform initial data processing tasks and make decisions based on predefined rules or machine learning models. Data requiring further analysis or storage is forwarded to cloud servers for processing. Cloud servers execute complex computations, store large datasets, and provide services such as data analytics, machine learning, and storage management.
- Dynamic resource allocation: One of the key features of the fog-cloud system model is dynamic resource allocation. Resources, including computational power, memory, and storage, are allocated based on the requirements of IoT applications and the availability of resources at fog nodes and cloud servers. This dynamic allocation ensures optimal utilization of resources while meeting the QoS requirements of IoT applications.

In summary, the system model for a fog-cloud computing environment integrates fog nodes, cloud servers, IoT devices, communication infrastructure, and dynamic resource allocation mechanisms to provide efficient and scalable processing of IoT data. This architecture enables real-time data processing, reduces latency, conserves bandwidth, and enhances the overall performance of IoT applications.

Problem formulation

The problem of task scheduling in a fog-cloud computing environment involves assigning tasks to computational nodes in a manner that minimizes makespan (total completion time) and energy consumption while considering the distinct characteristics of fog and cloud nodes. This can be mathematically formulated as follows:

Let $T = \{t_1, t_2, \dots, t_n\}$ represent the set of tasks to be scheduled, $F = \{f_1, f_2, \dots, f_m\}$ denote the set of fog nodes, and $C = \{c_1, c_2, \dots, c_k\}$ represent the set of cloud nodes. Each node has specific parameters associated with its capabilities:

1. Fog Nodes ($f_i \in F$): These are characterized by proximity to IoT devices, limited computational capacity C_{f_i} , energy consumption rate E_{f_i} , and low latency L_{f_i} .
2. Cloud Nodes ($c_j \in C$): These offer high computational power C_{c_j} , higher energy consumption E_{c_j} , and higher latency L_{c_j} due to their centralized nature.

The decision variable for task assignment is represented as:

$$x_{ij} = \begin{cases} 1 & \text{if task } t_i \text{ is assigned to fog node } f_j \\ 0 & \text{otherwise} \end{cases}, \quad y_{ik} = \begin{cases} 1 & \text{if task } t_i \text{ is assigned to cloud node } c_k \\ 0 & \text{otherwise} \end{cases}.$$

The optimization problem seeks to minimize the following objectives:

1. Makespan: The maximum completion time across all tasks assigned to fog and cloud nodes:

$$\text{Minimize } Z = \max \left(\sum_{i=1}^n x_{ij} \frac{P_i}{C_{f_j}}, \sum_{i=1}^n y_{ik} \frac{P_i}{C_{c_k}} \right),$$

where P_i represents the processing requirement of task t_i .

2. Energy consumption: The total energy consumed by fog and cloud nodes for executing tasks:

$$\text{Minimize } E = \sum_{i=1}^n \left(\sum_{j=1}^m E_{f_j} \cdot x_{ij} + \sum_{k=1}^k E_{c_k} \cdot y_{ik} \right).$$

The problem is subject to the following constraints:

1. Task assignment: Each task must be assigned to exactly one node (fog or cloud):

$$\sum_{j=1}^m x_{ij} + \sum_{k=1}^k y_{ik} = 1, \quad \forall i.$$

2. Node capacity:

$$\sum_{i=1}^n P_i \cdot x_{ij} \leq C_{f_j}, \quad \forall j, \quad \sum_{i=1}^n P_i \cdot y_{ik} \leq C_{c_k}, \quad \forall k.$$

3. Latency constraints:

$$L_{f_j} \cdot x_{ij} + L_{c_k} \cdot y_{ik} \leq L_{\max}, \quad \forall i,$$

where $L_{\max}$ represents the maximum allowable latency for tasks.

4. Non-negativity:

$$x_{ij}, y_{ik} \in \{0, 1\}, \quad \forall i, j, k.$$

This formulation explicitly differentiates between fog and cloud nodes by incorporating their unique characteristics into the parameters for capacity, energy consumption, and latency. The proposed task scheduling algorithm, TS-GWO, is applied to solve this problem, ensuring efficient resource allocation across fog and cloud environments while meeting Quality of Service (QoS) requirements.

GWO algorithm fundamentals

The Grey Wolf Algorithm (GWO) is a nature-inspired optimization technique that emulates the hunting behavior of grey wolves^{1,19,20}. In GWO, candidate solutions are depicted as members of a wolf pack, and their positions are iteratively adjusted to enhance a given objective function. The algorithm leverages the hierarchical structure and social dynamics within wolf packs, with alpha, beta, and delta wolves taking leadership roles. GWO involves six key steps: initialization of wolf positions, selection of leader wolves based on fitness, updating positions using specialized equations reflecting different hunting strategies, ensuring boundary constraints, evaluating fitness at new positions, and updating leader wolves accordingly. This iterative process continues until termination criteria, such as convergence or a maximum number of iterations, are met. Overall, GWO effectively explores and exploits the search space to optimize the objective function.

Grey Wolf Algorithm depicted in Algorithm 1 and explained in the following text.

-
- 1: **Initialization:** Initialize the positions of the wolves randomly within the search space.
 - 2: **Leader Selection:** Identify the alpha, beta, and delta wolves based on their fitness values.
 - 3: **Update Positions:**
 - a. Search for Prey (Alpha Wolf):
Calculate $\vec{D}_{\alpha} = |\vec{C}_{\alpha} \cdot \vec{X}_{\alpha} - \vec{X}|$
Update $\vec{X}_{\text{new}} = \vec{X}_{\alpha} - \vec{A} \cdot \vec{D}_{\alpha}$
 - b. Chase Prey (Beta Wolf):
Calculate $\vec{D}_{\beta} = |\vec{C}_{\beta} \cdot \vec{X}_{\beta} - \vec{X}|$
Update $\vec{X}_{\text{new}} = \vec{X}_{\beta} - \vec{A} \cdot \vec{D}_{\beta}$
 - c. Random Exploration (Delta Wolf):
Calculate $\vec{D}_{\delta} = |\vec{C}_{\delta} \cdot \vec{X}_{\delta} - \vec{X}|$
Update $\vec{X}_{\text{new}} = \vec{X}_{\delta} - \vec{A} \cdot \vec{D}_{\delta}$
 - 4: **Boundary Handling:** Ensure that the new positions remain within the search space.
 - 5: **Fitness Evaluation:** Evaluate the fitness of the wolves at their new positions.
 - 6: **Leader Update:** Update the alpha, beta, and delta wolves based on their fitness values.
 - 7: **Termination:** Repeat steps 3-6 until a termination criterion is met.
-

Algorithm 1. Grey wolf algorithm.

The proposed task scheduling with grey wolf optimization (TS-GWO)

The Task Scheduling with Grey Wolf Optimization (TS-GWO) algorithm is designed to address the problem of task scheduling in a hybrid fog-cloud computing environment. It minimizes the combined objectives of *makespan* and *energy consumption*, optimizing the assignment of tasks to fog and cloud nodes, as presented in Algorithm 2. Below, the detailed steps of the algorithm are described.

Require: Number of wolves N , maximum iterations T_{max} , task set T , node set M

Ensure: Optimal task-to-node assignment minimizing the objective function

- 1: Initialize positions of N grey wolves randomly within the solution space
- 2: Evaluate the fitness $F(x)$ of each grey wolf using the objective function (Eq. 6.1)
- 3: Identify the alpha (α), beta (β), and delta (δ) wolves based on fitness
- 4: **for** $t = 1$ to T_{max} **do**
- 5: **for** each grey wolf i **do**
- 6: Compute A (amplitude coefficient) and D (distance vector)
- 7: Update the position of wolf i using:

$$x_i(t+1) = x_i(t) - A \cdot D$$

- 8: **end for**
- 9: Perform **exploration** using random perturbations:

$$x_i(t+1) = x_i(t) + \Delta_{\text{exp}}$$

- 10: Perform **exploitation** by guiding solutions towards alpha, beta, and delta wolves:

$$x_i(t+1) = x_i(t) + \Delta_{\text{exp}}$$

- 11: Evaluate fitness of updated positions and update α , β , and δ
- 12: **end for**
- 13: Return the best solution (α) found

Algorithm 2. The proposed task scheduling with grey wolf optimization (TS-GWO).

Initialization

Representation of solutions: Each solution represents a mapping of tasks (T) to computing nodes, comprising fog nodes (F) and cloud nodes (C). A binary decision variable x_{ij} is used to indicate whether a task t_i is assigned to a node n_j .

Fog and cloud nodes are differentiated as follows:

- Fog nodes (F): Characterized by lower computational capacity and energy consumption but provide low latency.
- Cloud nodes (C): Offer higher computational capacity but consume more energy and have higher latency.

Population initialization: The algorithm initializes a population of N grey wolves randomly within the solution space, where each wolf represents a unique task-to-node assignment.

Fitness evaluation: The objective function $F(x)$, defined as:

$$F(x) = \alpha \cdot Z + \beta \cdot E, \quad (1)$$

is used to evaluate the fitness of each solution. Here, Z represents the *makespan*, E denotes the *energy consumption*, and α and β are weighting factors. The best-performing wolves are identified as *alpha* (α), *beta* (β), and *delta* (δ), which guide the optimization process.

Iterative optimization

The TS-GWO algorithm iteratively balances exploration (discovering new task assignments) and exploitation (refining promising assignments).

Exploration phase

Purpose: The exploration phase ensures a diverse search of the solution space to prevent premature convergence.

Mechanism: Each grey wolf's position is updated using a random exploration vector Δ_{exp} :

$$x_i(t+1) = x_i(t) + \Delta_{\text{exp}}, \quad (2)$$

where $x_i(t)$ and $x_i(t+1)$ represent the current and updated positions of the i -th grey wolf, respectively. This operation introduces randomness in task assignments, allowing the algorithm to explore untested configurations of fog and cloud nodes.

Exploitation phase

Purpose: The exploitation phase refines the search around promising solutions identified by the alpha, beta, and delta wolves.

Mechanism: Each grey wolf adjusts its position toward the alpha, beta, and delta wolves:

$$x_i(t+1) = x_i(t) + \Delta_{\text{exp}}, \quad (3)$$

where the adjustment guides the wolf's position toward regions of the solution space with higher fitness values.

Position update

The position update equation for each grey wolf is given by:

$$x_i(t+1) = x_i(t) - A \cdot D, \quad (4)$$

where A is the amplitude coefficient controlling the intensity of adjustments, and D is the distance vector between the wolf's current position and the target position. These parameters dynamically adapt as the algorithm progresses, balancing exploration and exploitation.

Node-specific assignments

- Fog node assignments: Tasks requiring low latency are prioritized for assignment to fog nodes, constrained by their limited capacity (C_j^{fog}) and energy consumption (E_j^{fog}).
- Cloud node assignments: Computationally intensive tasks or those exceeding fog node capacity are assigned to cloud nodes, leveraging their higher capacity (C_j^{cloud}) despite increased energy consumption (E_j^{cloud}).

Fitness evaluation and solution update

After each position update, the fitness of all grey wolves is reevaluated. The best-performing wolves are reassigned as the alpha, beta, and delta wolves, ensuring the solution set evolves toward better task-to-node assignments.

Termination and output

The algorithm continues until a termination condition is met, such as:

- Maximum number of iterations (T_{max}) reached.
- Convergence of the solution set to a stable fitness value.

The best solution (α) is returned, providing the optimal task-to-node mapping that minimizes the combined objectives of makespan and energy consumption.

Complexity analysis

The computational complexity of the proposed TS-GWO algorithm is determined by analyzing its key operations. During initialization, N grey wolves are randomly positioned in the solution space, with a complexity of $O(N \cdot n_T)$. Fitness evaluation, which involves calculating makespan and energy consumption for n_T tasks and n_M nodes, contributes $O(N \cdot n_T \cdot n_M)$ per iteration. Updating the alpha, beta, and delta wolves requires $O(N \log N)$, while position updates for all wolves add $O(N \cdot n_T)$ per iteration. These steps are repeated for T_{max} iterations, resulting in a total computational complexity of:

$$O(T_{\text{max}} \cdot N \cdot n_T \cdot n_M).$$

This indicates that the runtime of TS-GWO scales with the number of iterations, wolves, tasks, and nodes, reflecting its computational demands in large-scale fog-cloud environments.

Key features and advantages

- Dynamic resource allocation: Efficiently balances latency-sensitive tasks on fog nodes and computationally intensive tasks on cloud nodes.
- Objective balancing: Simultaneously minimizes makespan and energy consumption.
- Adaptive search: Dynamically balances exploration and exploitation through parameter adjustments.

Experiment and results

This section delves into the evaluation of the proposed approach by conducting a series of experiments. To begin, we provide an overview of the experimental configuration and elaborate on the characteristics of the dataset employed in these trials. Subsequently, we define the metrics employed for evaluating our proposal. Finally, we present and analyze the experimental results, along with providing insights into the key findings observed.

Experimental setup

Performance evaluations and comparisons with established scheduling algorithms were executed using MATLAB R2018b. The experiments were conducted on a DELL PC featuring an Intel Core i5-2430 2.40 GHz CPU and 4 GB of memory, operating on Windows 10 Professional 64-bit. The experimental arrangement encompassed a fog-cloud environment comprising two data centers, four host machines, and 25 VMs configured with diverse

Parameter	Value
Number of clients	Ranged from 50 to 100
Number of data centers	2
Number of hosts	4
Bandwidth	10 Gb/s
Storage capacity	1 TB
RAM size	16 GB
Architecture	X86
Policy type	Time shared

Table 2. Cloud entity parameters.

Parameter	Value
Number of VMs	25
CPU capacity	Range: 100 to 5000 MIPS
RAM size	1 GB
Storage	20 GB
VMM	Xen
Bandwidth	1 Gb/s
Policy type	Time shared
Number of CPUs	1

Table 3. Virtual machine parameters.

Parameter	Value
Length of the task	Range: 1000 to 20000 MI
Number of tasks	Range: 200 to 1000
File size	Range: 300 to 600 MB
Output size	Range: 300 to 600 MB

Table 4. Characteristics of the synthetic workload.

Description of workload	Duration	CPU allocation	Number of users	Number of tasks	Resource utilization
HPC2N	July 2002 to January 2006	240	257	202,871	60.1%
NASA iPSC	October 1993 to December 1993	128	69	18,239	46.6%

Table 5. Characteristics of the actual workload.

specifications. The experimental parameters were specified in Tables 2-3. This setup enabled comprehensive evaluations of scheduling algorithms under varied conditions, considering different numbers of clients, resource capacities, and VM configurations in the fog-cloud environment.

The task sizes of 1000, 2000, 3000, 4000, and 5000 were chosen to comprehensively evaluate the scalability and robustness of the proposed TS-GWO algorithm in handling varying workload intensities in fog-cloud environments. These values represent diverse task loads, ranging from moderate to high, which are typical in real-world scenarios involving IoT applications. By testing across this range, we ensure that the algorithm's performance can be assessed under different levels of computational demand, reflecting its adaptability and efficiency. Additionally, these task sizes align with benchmark datasets used in related studies, facilitating a meaningful comparison with existing methods and providing a standardized basis for evaluation.

For the experiments, we employed a combination of synthetic and real datasets to assess the effectiveness of the proposed TSGWO algorithm. The synthetic workload consisted of 1000 tasks generated using a uniform distribution, with task lengths ranging from 1000 to 20,000 MI. Additionally, we utilized real-world datasets sourced from the 'Parallel Workload Archive', specifically the HPC2N and NASA Ames iPSC/860 datasets. Details of both the synthetic and parallel workloads are presented in Tables 4 and 5, respectively.

To ensure the robustness of our findings, we conducted each experiment independently and repeated it 30 times. Subsequently, we reported the average values obtained from these repetitions. Specific parameter settings for the algorithms utilized in this study are detailed in Table 6. These experiments allowed for thorough

Method	Values
AHA	$a \rightarrow [0,1], R \rightarrow [0,1], b \rightarrow [0,1]$
MPA	$P \rightarrow 0.5, FADs \rightarrow 0.2$
GJO	$c1 \rightarrow 0.5, r \rightarrow [0,1], rl \rightarrow [0,1]$
DJSD	$\eta \rightarrow 4.0, \gamma \rightarrow 0.1$
AOA	$MOP \text{ Min} \rightarrow 0.2, MOP \text{ Max} \rightarrow 1, \mu \rightarrow 0.499, \alpha \rightarrow 5$
CSA	$\alpha \rightarrow 4, \gamma \rightarrow 2, \beta \rightarrow 3, \rho \rightarrow 1$
TS-GWO	$R \rightarrow [0,1], a \rightarrow [0,1], b \rightarrow [0,1], c1 \rightarrow 0.5, r \rightarrow [0,1], rl \rightarrow [0,1]$
AOS	Foton rate $\rightarrow 0.1$, Total number of layers $\rightarrow 5$

Table 6. Configurations for the methods' parameters.

evaluations of the TSGWO algorithm's performance and its comparison with other scheduling algorithms under various workload conditions.

Performance metrics

To evaluate the performance of the proposed TS-GWO algorithm in task scheduling for fog-cloud computing environments, we focus on two critical performance metrics: *makespan* and *energy consumption*. These metrics are mathematically defined as follows:

Makespan (Z)

The makespan represents the maximum time required to complete all assigned tasks across the available fog and cloud computing nodes³. It ensures balanced workload distribution and minimizes task completion time. Mathematically, it is expressed as:

$$Z = \text{Max} \left(\frac{1}{C_k} \sum_{i=1}^n \sum_{j=1}^m P_j \cdot x_{ij} \right), \quad (5)$$

where: n is the total number of tasks. m is the total number of computing nodes (fog or cloud). P_j is the processing time for task t_i on computing node n_j . x_{ij} is the binary decision variable indicating whether task t_i is assigned to node n_j ($x_{ij} = 1$) or not ($x_{ij} = 0$). C_k is the processing capacity of the computing node k . $\text{Max}()$ is the operator to find the maximum completion time across all nodes.

This metric ensures that the completion time for the most heavily loaded node is minimized.

Energy consumption (E)

Energy consumption measures the total energy used by all computing nodes for executing the assigned tasks¹. It reflects the efficiency of resource usage and is expressed as:

$$E = \sum_{i=1}^n \sum_{j=1}^m E_j \cdot x_{ij}, \quad (6)$$

where: E_j is the energy consumption per unit time for node n_j when processing a task. x_{ij} is the binary decision variable as defined above.

The summation aggregates the energy consumed for all tasks across all computing nodes. This metric highlights the algorithm's capability to reduce overall energy usage, crucial for energy-efficient fog-cloud systems.

Constraints

The metrics are evaluated under the following constraints to ensure valid and efficient task scheduling:

1. Task assignment: Each task t_i must be assigned to exactly one computing node:

$$\sum_{j=1}^m x_{ij} = 1, \quad \forall i \quad (7)$$

2. Node capacity: The total workload assigned to a node must not exceed its processing capacity:

$$\sum_{i=1}^n P_j \cdot x_{ij} \leq C_k, \quad \forall k \quad (8)$$

3. Non-negativity: The decision variable must satisfy:

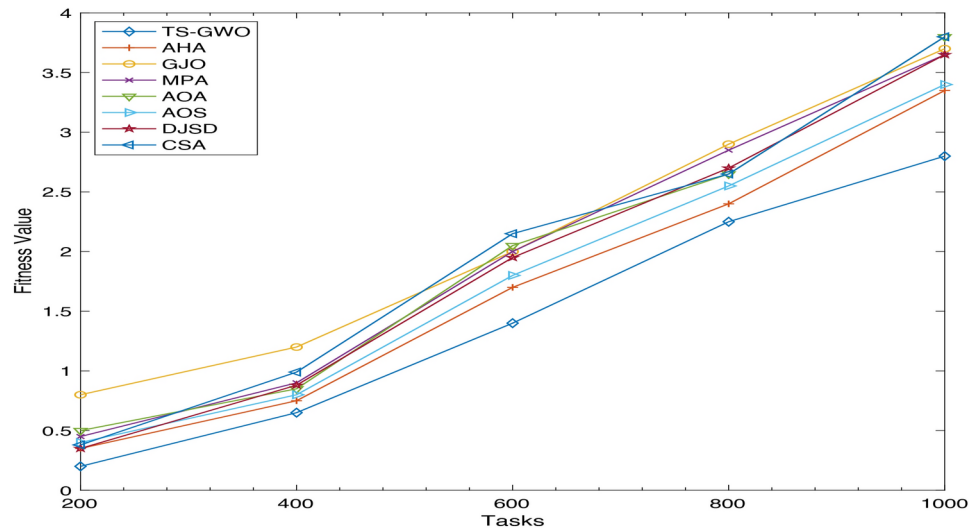


Fig. 2. Synthetic data fitness analysis.

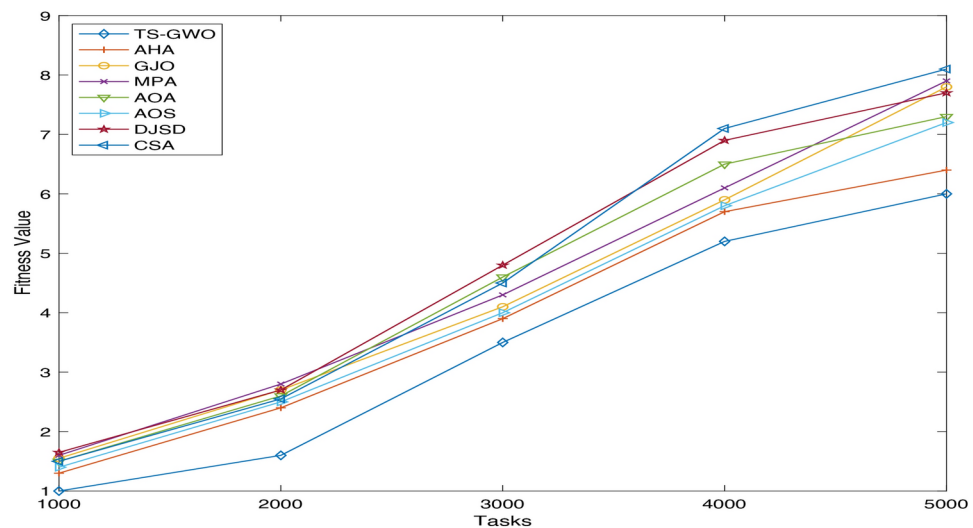


Fig. 3. Real data NASA iPSC fitness analysis.

$$x_{ij} \geq 0, \quad \forall i, j \quad (9)$$

These metrics form the foundation for evaluating the effectiveness of TS-GWO in optimizing task scheduling in fog-cloud environments, addressing both computational efficiency and energy consumption.

The main goal of this investigation is to lower the overall energy consumption while also minimizing makespan, with the aim of improving the overall efficiency of the system.

Results and analysis

This section presents a comprehensive analysis of the evaluation results obtained from the proposed TS-GWO algorithm, highlighting its performance across several benchmarked algorithms and various workload traces.

To assess the performance of the TS-GWO, it was subjected to a comparative analysis against several state-of-the-art algorithms: AHA²⁴, GJO²⁵, Atomic Orbital Search (AOS)²⁶, AOA¹¹, MPA²⁷, Dynamic Jellyfish Simulated Disruption (DJSD)²⁸, and the Chameleon Swarm Algorithm (CSA)²⁹. Table 6 outlines the parameter settings for both TS-GWO and the comparison algorithms. For consistency, common parameters, such as the number of iterations (1000) and population size (50), were maintained across all experiments, with each experiment repeated 30 times, and the results averaged. Additionally, the parameter ϕ was set to 0.7 in all cases.

The performance of the TS-GWO algorithm is evaluated across various datasets and real-world applications, with performance curves depicted in Figs. 2, 3 and 4. Specifically, Figure 2 presents the performance curve for synthetic workloads, where TS-GWO consistently outperforms all other algorithms across all instances. Similarly, Figure 3 demonstrates TS-GWO's superior performance on the NASA Ames iPSC/860 dataset. Figure

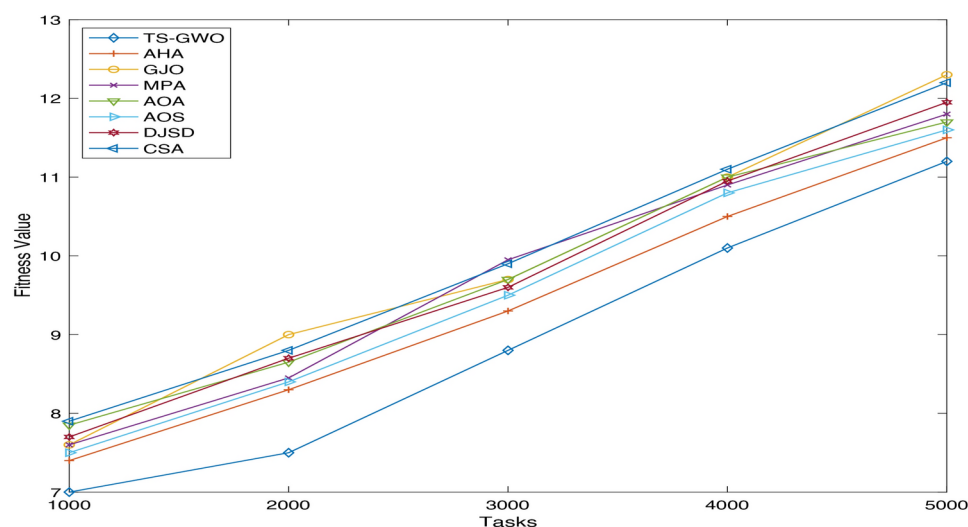


Fig. 4. Real data HPC2N fitness analysis.

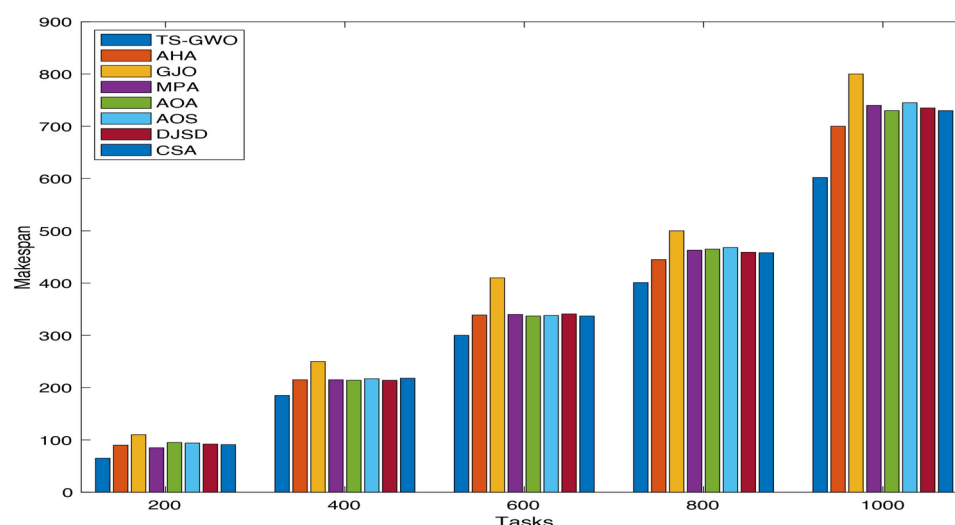


Fig. 5. Synthetic data makespan analysis.

4 shows TS-GWO's performance across varying task volumes (1000 to 5000 tasks) for the HPC2N workload, where it consistently surpasses the performance of the seven other techniques. In terms of fitness function values, TS-GWO outperforms the competing algorithms, with GWO showing the best overall performance, followed by AHA and AOS.

In terms of makespan, Figs. 5, 6, 7 illustrate the comparative results of TS-GWO and other algorithms. Figure 5 clearly demonstrates TS-GWO's superior average makespan when compared to the GJO algorithm and others on synthetic datasets. Figure 6 further validates TS-GWO's superiority in terms of makespan on the NASA Ames iPSC workload. Similarly, Fig. 7 confirms TS-GWO's outperformance across the HPC2N workload. The average improvements in makespan across various workloads are significant, with improvements ranging from 9.89% to 40.91% for synthetic datasets, 4.02% to 34.94% for NASA iPSC workloads, and 2.91% to 46.15% for HPC2N workloads when compared to the other algorithms.

Figures 8, 9, 10 present a detailed comparison of the total energy consumption across algorithms. In all cases, TS-GWO consistently exhibits the lowest energy consumption, outperforming other algorithms in both synthetic and real-world datasets. In particular, Figure 8 highlights TS-GWO's superior energy efficiency on synthetic datasets. Likewise, Figure 9 demonstrates its energy efficiency across the NASA Ames iPSC workload, and Figure 10 further confirms TS-GWO's best energy consumption performance on the HPC2N workload. The average improvements in energy consumption are noteworthy, with TS-GWO achieving reductions ranging from 7.94% to 28.57% for synthetic workloads, 3.54% to 14.62% for NASA iPSC workloads, and 2.16% to 9.11% for HPC2N workloads compared to other methods.

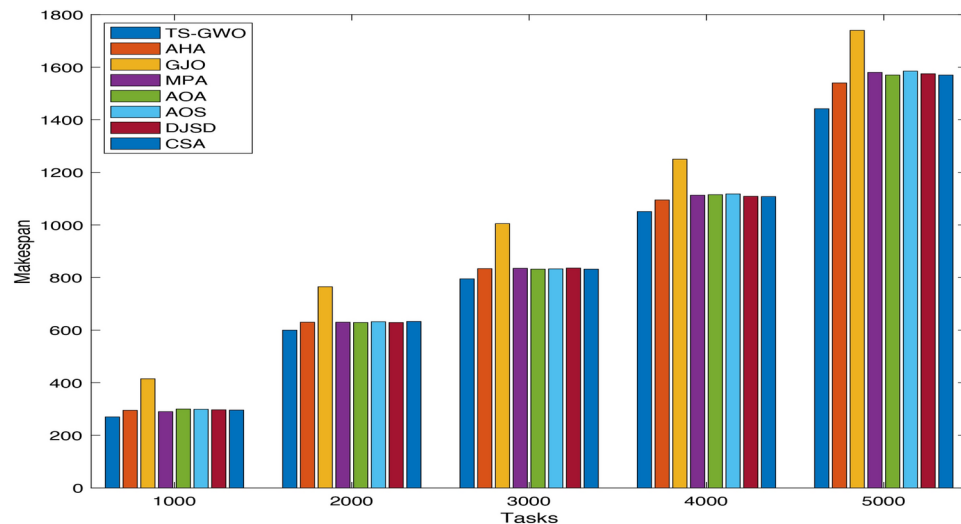


Fig. 6. Real data NASA iPSC makespan analysis.

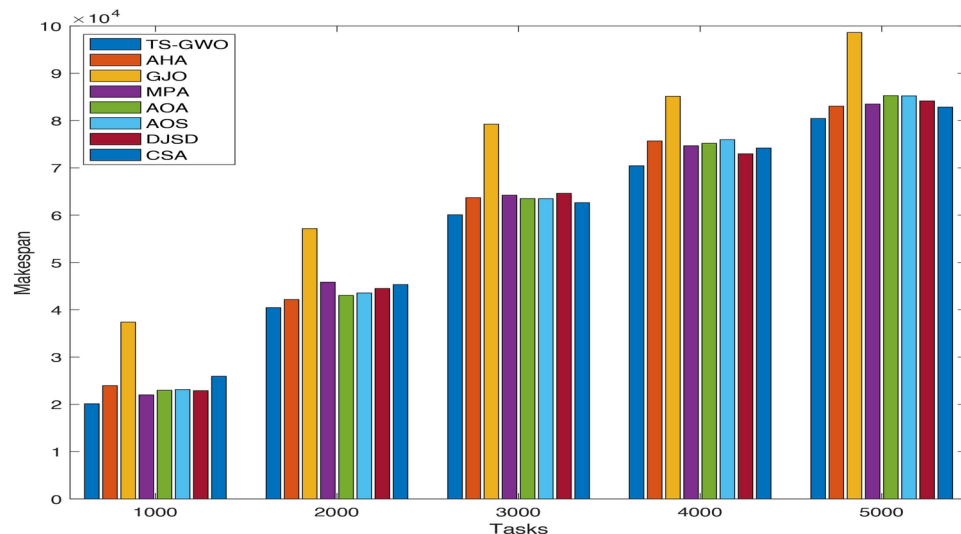


Fig. 7. Real data HPC2N makespan analysis.

The improvement in makespan and other parameters in our proposed TS-GWO approach can be attributed to its enhanced ability to balance exploration and exploitation through the modified GWO algorithm. The new operators in TS-GWO allow for better search space exploitation, resulting in more optimal task-to-node assignments. This leads to reduced makespan and improved energy efficiency, particularly under the dynamic conditions of fog-cloud environments where task and resource distributions can vary.

It can be noted that TS-GWO has shown promising results across a variety of datasets, including synthetic and real-world scenarios like NASA Ames iPSC and HPC2N workloads. However, while it performs well in many cases, the approach is particularly suited to environments where the task scheduling needs to consider both makespan and energy consumption as primary objectives. For environments with highly unpredictable workloads or significantly larger-scale systems, further optimization or adjustments to the algorithm may be necessary.

However, TS-GWO may face challenges in extremely large-scale scenarios with complex interdependencies between tasks, where the computational cost of fitness evaluations may become prohibitive. Furthermore, the algorithm's performance could be sensitive to the specific parameter tuning, which may require domain-specific adjustments in some cases. We plan to address these limitations in future work by exploring multi-objective optimization and hybrid approaches that can better handle such complex environments.

In summary, the results demonstrate that TS-GWO significantly improves both makespan and energy consumption compared to established scheduling algorithms across various workload traces. TS-GWO consistently achieves lower makespan and energy consumption, thus optimizing task completion time and resource usage more effectively. These findings validate TS-GWO as a robust optimization technique for fog-

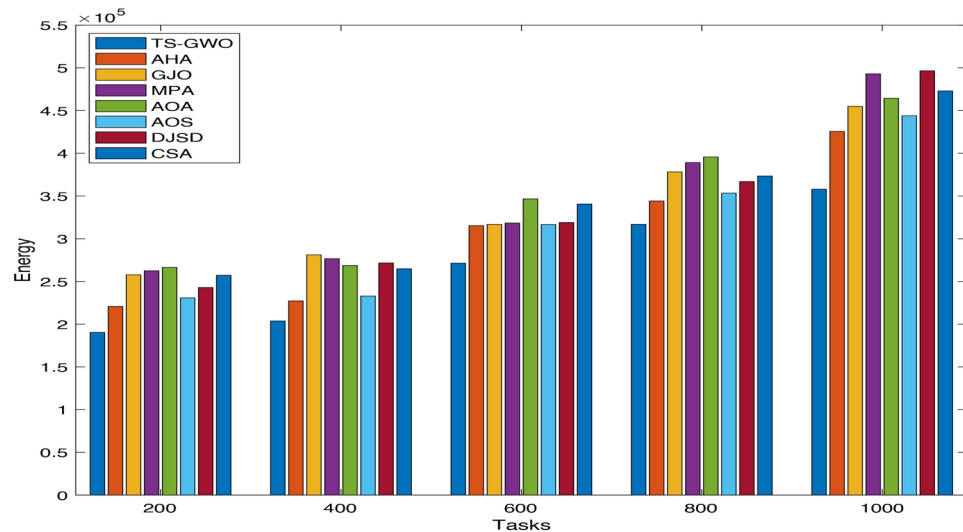


Fig. 8. Synthetic data energy consumption analysis.

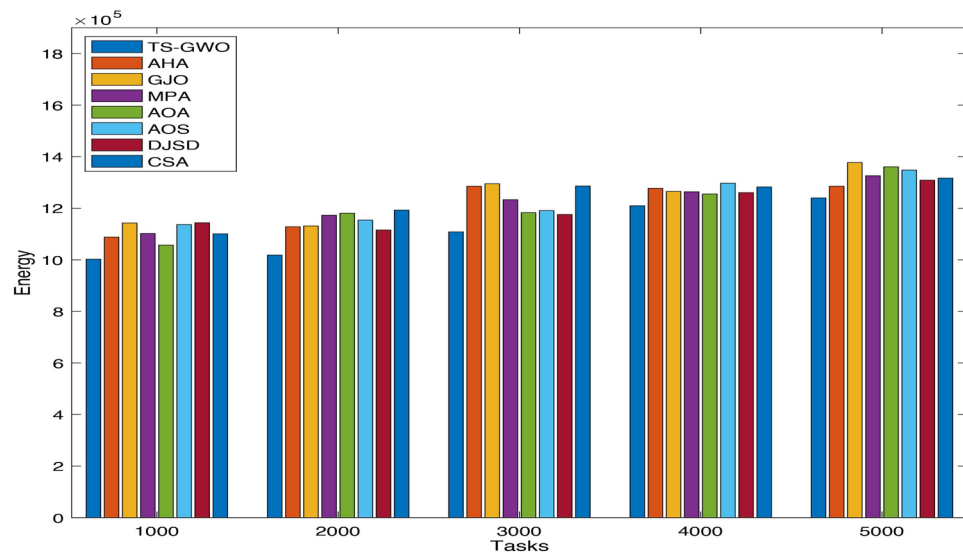


Fig. 9. Real data NASA iPSC energy consumption analysis.

cloud computing environments, where both performance and energy efficiency are crucial. The experimental analysis also highlights the potential for further optimization by integrating TS-GWO with complementary algorithms, such as AHA and GJO, to enhance search capabilities and generate even more optimal solutions across diverse scenarios. The results clearly underscore TS-GWO's potential for practical deployment and future research applications in distributed computing environments.

Conclusion

In conclusion, this paper introduces the modified Grey Wolf Optimization (TS-GWO) algorithm as a solution to task scheduling challenges in fog-cloud computing environments. Through extensive experimentation and comparative analysis, we have demonstrated the effectiveness of TS-GWO in minimizing makespan and optimizing energy consumption compared to existing scheduling algorithms. The results show that TS-GWO outperforms other methods across various workload traces, including synthetic datasets and real-world applications like NASA Ames iPSC and HPC2N workloads. Specifically, TS-GWO achieves improvements in makespan ranging from 9.89% to 40.91% for synthetic workloads, 4.02% to 34.94% for NASA Ames iPSC workloads, and 2.91% to 46.15% for HPC2N workloads. Additionally, TS-GWO demonstrates reductions in energy consumption ranging from 7.94% to 28.57% for synthetic workloads, 3.54% to 14.62% for NASA Ames iPSC workloads, and 2.16% to 9.11% for HPC2N workloads.

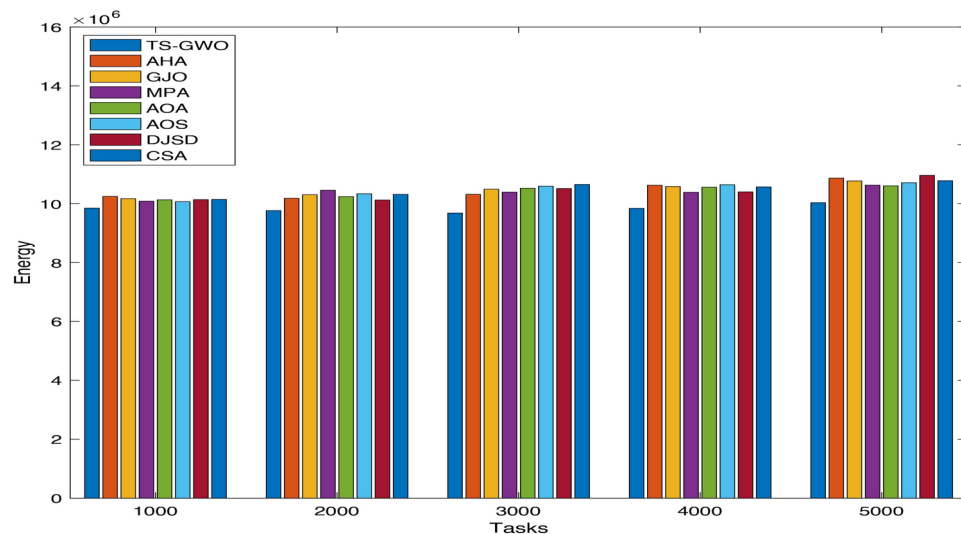


Fig. 10. Real data HPC2N energy consumption analysis.

These results highlight TS-GWO's ability to significantly reduce task completion times while demonstrating superior energy efficiency. This makes TS-GWO a promising solution for optimizing fog-cloud systems, where efficient resource utilization is critical. The superior performance of TS-GWO across different datasets underscores its potential as an effective optimization tool for distributed computing environments, offering substantial improvements in both makespan and energy consumption.

While our evaluation focused on makespan and energy consumption, we recognize that other metrics, such as scheduling overhead, throughput, and latency, are essential for a comprehensive performance assessment. These aspects will be explored in future work, which will aim to provide a more complete evaluation of TS-GWO's capabilities. Furthermore, extending TS-GWO to address multi-objective optimization, particularly in the context of IoT applications, is a promising direction. This extension would allow the algorithm to optimize multiple conflicting objectives, such as task completion time, energy usage, and resource allocation, providing a more holistic solution to scheduling challenges. Additionally, exploring hybrid approaches that combine TS-GWO with other optimization techniques, such as PSO or GA, could further enhance the algorithm's exploration and exploitation capabilities.

Despite its promising results, there are limitations to the current TS-GWO implementation. Scalability with larger systems remains a challenge, as the algorithm may experience performance degradation when applied to very large-scale distributed environments. Furthermore, the algorithm's computational overhead, which includes the complexity of its iterative process and fitness evaluations, could limit its applicability for real-time task scheduling in time-sensitive applications. To address this concern, we plan to evaluate the scheduling overhead of TS-GWO in future work and compare it with other approaches to provide a more comprehensive performance analysis. Additionally, the sensitivity of TS-GWO to parameter selection is an area that warrants further investigation. Dynamic parameter adjustment could improve its robustness and adaptability to various task scheduling scenarios. Another avenue for future research is exploring how TS-GWO can be enhanced to handle dynamic task scheduling, where tasks and resources change over time due to external factors like network failures or workload variations.

In summary, the experimental results confirm the effectiveness of TS-GWO in optimizing task scheduling in fog-cloud computing environments. Moving forward, we aim to refine the algorithm to address its current limitations and explore its broader applicability to other optimization problems, such as knapsack problems and job scheduling. We also plan to extend TS-GWO's use to multi-objective optimization scenarios. With further development, the future potential of TS-GWO is immense, and we are excited about its capability to improve scheduling in distributed systems while addressing its limitations.

Data availability

All related data is available in the paper.

Received: 23 June 2024; Accepted: 23 April 2025

Published online: 27 April 2025

References

1. Saif, F. A., Latip, R., Hanapi, Z. M. & Shafinah, K. Multi-objective grey wolf optimizer algorithm for task scheduling in cloud-fog computing. *IEEE Access* **11**, 20635–20646. <https://doi.org/10.1109/ACCESS.2023.3241240> (2023).
2. Tong, Z., Chen, H., Deng, X., Li, K. & Li, K. A scheduling scheme in the cloud computing environment using deep q-learning. *Inf. Sci.* **512**, 1170–1191. <https://doi.org/10.1016/j.ins.2019.10.035> (2020).

3. Attiya, I., Al-qaness, M. A., Abd Elaziz, M. & Aseeri, A. O. Boosting task scheduling in iot environments using an improved golden jackal optimization and artificial hummingbird algorithm. *AIMS Math.* **9**, 847–867. <https://doi.org/10.3934/math.2024043> (2024).
4. Abd Elaziz, M., Abualigah, L. & Attiya, I. Advanced optimization technique for scheduling iot tasks in cloud-fog computing environments. *Future Gener. Comput. Syst.* **124**, 142–154. <https://doi.org/10.1007/s11831-020-09481-7> (2021).
5. Mtshali, M., Kobo, H., Dlamini, S., Adigun, M. & Mudali, P. Multi-objective optimization approach for task scheduling in fog computing. In *2019 International Conference on Advances in Big Data, Computing and Data Communication Systems (icABCD)*. 1–6. <https://doi.org/10.1109/ICABCD.2019.8851038> (IEEE, 2019).
6. Chen, Z.-G. et al. Multiobjective cloud workflow scheduling: A multiple populations ant colony system approach. *IEEE Trans. Cybern.* **49**, 2912–2926. <https://doi.org/10.1109/TCYB.2018.2832640> (2018).
7. Nguyen, B. M., Thi Thanh Binh, H., The Anh, T. & Bao Son, D. Evolutionary algorithms to optimize task scheduling problem for the iot based bag-of-tasks application in cloud-fog computing environment. *Appl. Sci.* **9**, 1730. <https://doi.org/10.3390/app9091730> (2019).
8. Boveiri, H. R., Khayami, R., Elhoseny, M. & Gunasekaran, M. An efficient swarm-intelligence approach for task scheduling in cloud-based internet of things applications. *J. Ambient Intell. Hum. Comput.* **10**, 3469–3479. <https://link.springer.com/article/10.1007/s12652-018-1071-1> (2019).
9. Yang, X. & Rahmani, N. Task scheduling mechanisms in fog computing: Review, trends, and perspectives. *Kybernetes* **50**, 22–38. <https://doi.org/10.1108/K-10-2019-0666> (2020).
10. Yang, M. et al. A multi-objective task scheduling method for fog computing in cyber-physical-social services. *IEEE Access* **8**, 65085–65095. <https://doi.org/10.1109/ACCESS.2020.2983742> (2020).
11. Abualigah, L., Diabat, A., Mirjalili, S., Abd Elaziz, M. & Gandomi, A. H. The arithmetic optimization algorithm. *Comput. Methods Appl. Mech. Eng.* **376**, 113609. <https://doi.org/10.1016/j.cma.2020.113609> (2021).
12. Kaur, R., Sachdeva, M. & Kumar, G. Nature inspired feature selection approach for effective intrusion detection. *Indian J. Sci. Technol.* **9**, 1–9. <https://doi.org/10.17485/ijst/2016/v9i42/101555> (2016).
13. Thakur, K. & Kumar, G. Nature inspired techniques and applications in intrusion detection systems: Recent progress and updated perspective. *Arch. Comput. Methods Eng.* **28**, 2897–2919. <https://doi.org/10.1007/s11831-020-09481-7> (2021).
14. Ghobaei-Arani, M., Sourai, A., Safara, F. & Norouzi, M. An efficient task scheduling approach using moth-flame optimization algorithm for cyber-physical system applications in fog computing. *Trans. Emerg. Telecommun. Technol.* **31**, e3770. <https://doi.org/10.1002/ett.3770> (2020).
15. Abualigah, L. et al. Ts-gwo: Iot tasks scheduling in cloud computing using grey wolf optimizer. In *Swarm Intelligence for Cloud Computing*. 127–152. <https://doi.org/10.1201/9780429020582-5> (Chapman and Hall/CRC, 2020).
16. Zeng, D., Gu, L., Guo, S., Cheng, Z. & Yu, S. Joint optimization of task scheduling and image placement in fog computing supported software-defined embedded system. *IEEE Trans. Comput.* **65**, 3702–3712. <https://doi.org/10.1109/TC.2016.2536019> (2016).
17. Deng, Z., Yan, Z., Huang, H. & Shen, H. Energy-aware task scheduling on heterogeneous computing systems with time constraint. *IEEE Access* **8**, 23936–23950. <https://doi.org/10.1109/ACCESS.2020.2970166> (2020).
18. Paknejad, P., Khorsand, R. & Ramezanpour, M. Chaotic improved PICEA-g-based multi-objective optimization for workflow scheduling in cloud environment. *Future Gener. Comput. Syst.* **117**, 12–28. <https://doi.org/10.1016/j.future.2020.11.002> (2021).
19. Faris, H., Aljarah, I., Al-Betar, M. A. & Mirjalili, S. Grey wolf optimizer: A review of recent variants and applications. *Neural Comput. Appl.* **30**, 413–435. <https://link.springer.com/article/10.1007/s00521-017-3272-5> (2018).
20. Mirjalili, S., Mirjalili, S. M. & Lewis, A. Grey wolf optimizer. *Adv. Eng. Softw.* **69**, 46–61. <https://doi.org/10.1016/j.advengsoft.2013.12.007> (2014).
21. Yang, Y., Yang, B., Wang, S., Liu, W. & Jin, T. An improved grey wolf optimizer algorithm for energy-aware service composition in cloud manufacturing. *Int. J. Adv. Manuf. Technol.* **105**, 3079–3091. [https://link.springer.com/article/10.1007/10.1007\(2019\)](https://link.springer.com/article/10.1007/10.1007(2019)).
22. Jiang, T. & Zhang, C. Application of grey wolf optimization for solving combinatorial problems: Job shop and flexible job shop scheduling cases. *IEEE Access* **6**, 26231–26240. <https://doi.org/10.1109/ACCESS.2018.2833552> (2018).
23. Liu, L., Chang, Z., Guo, X., Mao, S. & Ristaniemi, T. Multiobjective optimization for computation offloading in fog computing. *IEEE Internet Things J.* **5**, 283–294. <https://doi.org/10.1109/JIOT.2017.2780236> (2017).
24. Zhao, W., Wang, L. & Mirjalili, S. Artificial hummingbird algorithm: A new bio-inspired optimizer with its engineering applications. *Comput. Methods Appl. Mech. Eng.* **388**, 114194. <https://doi.org/10.1016/j.cma.2021.114194> (2022).
25. Chopra, N. & Ansari, M. M. Golden jackal optimization: A novel nature-inspired optimizer for engineering applications. *Expert Syst. Appl.* **198**, 116924. <https://doi.org/10.1016/j.eswa.2022.116924> (2022).
26. Azizi, M. Atomic orbital search: A novel metaheuristic algorithm. *Appl. Math. Model.* **93**, 657–683. <https://doi.org/10.1016/j.apm.2020.12.021> (2021).
27. Faramarzi, A., Heidarinejad, M., Mirjalili, S. & Gandomi, A. H. Marine predators algorithm: A nature-inspired metaheuristic. *Expert Syst. Appl.* **152**, 113377. <https://doi.org/10.1016/j.eswa.2020.113377> (2020).
28. Attiya, I., Abualigah, L., Alshathri, S., Elsadek, D. & Abd Elaziz, M. Dynamic jellyfish search algorithm based on simulated annealing and disruption operators for global optimization with applications to cloud task scheduling. *Mathematics* **10**, 1894. <https://doi.org/10.3390/math10111894> (2022).
29. Braik, M. S. Chameleon swarm algorithm: A bio-inspired optimizer for solving engineering design problems. *Expert Syst. Appl.* **174**, 114685. <https://doi.org/10.1016/j.eswa.2021.114685> (2021).

Author contributions

D.A. and M.A. developed the paper idea, developed the simulation model, and conducted simulations, D.A. wrote the initial version of the paper. All authors reviewed the manuscript.

Declarations

Competing Interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to D.A.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2025